A Model for Entity Profile Management Through Inference in Trails

André Wagner and Jorge Luis Victória Barbosa

Universidade do Vale do Rio dos Sinos (UNISINOS)
Unidade Acadêmica de Pesquisa e Pós-Graduação
Av. Unisinos, 950, São Leopoldo - RS, Brazil

Abstract

Context-aware and ubiquitous systems present the challenge of implicitly collect relevant information about the users, and use this information to understand the user and predict user behaviour. This will allow the applications to adapt themselves to the user, thus avoiding to overflow the user with inquires and information. This article presents a model that allows applications to register users' actions and infer profile informations from these actions, using semantic interoperability and thus allowing different applications of different domains to be able to share informations and infer a unified profile.

1 Introdução

Atualmente, a maioria dos usuários interage com uma grande variedade de sistemas computacionais. Cada vez mais, existe a tendência do aumento da interconectividade entre estes sistemas, devido ao fato da internet estar cada vez mais presente e disponível em qualquer lugar. Assim, pesquisas recentes preveem a integração completa de todos estes sistemas em uma federação de sistemas computacionais, que podem ser descritos como Ambientes Inteligentes [5].

Uma visão para o conceito de ambientes inteligentes foi discutida por Weiser, onde ele apresentou um cenário em que o computador se tornaria invisível e indistinguível de seu ambiente [15]. Neste cenário, a interação entre humanos e computadores ocorreria não como a usual – onde o humano dedica 100% de sua atenção ao computador –, mas constituíra-se de pequenas interações rápidas, geralmente em locais e dispositivos distintos, muitas vezes utilizando a periferia da atenção do usuário [16] ou sem ele se dar conta de estar interagindo com um sistema computacional. Segundo o autor, "as tecnologias mais profundas são aquelas que desaparecem. Elas se entrelaçam no tecido do dia-a-dia até que se tornem indistinguíveis dele" [15]. Ele defende que o futuro da computação está em integrar o computador ao mundo real, e chama este novo paradigma de Computação Ubíqua.

No entanto, o aumento dos recursos computacionais disponíveis a qualquer momento e em qualquer lugar oferece o risco da sobrecarga de informações para o usuário. Para resolver este problema, foi criado o conceito de "tecnologia calma" [16]. Dentro desta ideia, a transmissão de informações entre o computador e o usuário devem ocorrer na "periferia" da atenção do usuário, movendo-se para o "centro" da atenção em momentos de urgência. Um exemplo é o motor de um carro: quando um motorista dirige um carro, sua atenção está focada na estrada, e ele mal percebe o som do motor. No entanto, se houver algum som diferente no motor (o que pode indicar algum problema), a atenção do usuário passa a focar-se no motor.

Para que um sistema seja minimamente intrusivo, é necessário que ele seja capaz de reconhecer a situação do usuário e seus arredores, e adapte seu comportamento de acordo com esta informação. Esta informação é chamada de contexto e sistemas capazes deste tipo de adaptação são chamados sensíveis a contexto [9, 4].

Pesquisas nesta área têm indicado que não apenas é interessante conhecer o contexto do usuário no momento em que um determinado serviço está sendo oferecido, mas que o histórico de contextos visitados pode ser uma informação valiosa na adaptação da aplicação ao usuário. Este histórico de contextos recebe o nome de Trilha [10]. Embora a trilha geralmente esteja vinculada à localização de um usuário, ela possui um conceito mais amplo, que inclui as atividades realizadas, aplicações usadas, conteúdos acessados – entre outros – dentro de um contexto e um período de tempo es-

pecífico [11].

Para que um aplicativo possa se adaptar a um usuário de forma eficaz, é importante que ele conheça não apenas o histórico de contextos visitados por um usuário, mas possa compreender quem o usuário é, isto é, quais são suas características. As aplicações deste tipo organizam seu conhecimento a respeito do usuário em perfis de usuário, os quais armazenam as informações em um formato baseado em um modelo de usuário específico [12]. Este conceito dá origem à Personalização.

Personalização é a disponibilização de produtos, serviços e informações adaptados individualmente. A tecnologia de personalização geralmente envolve programas que aprendam padrões, hábitos e preferências de um usuário [7]. O objetivo principal por detrás dos sistemas de personalização modernos é oferecer às entidades o que elas querem sem requisitar que elas informem isto de forma explícita [7]. Fica claro que este propósito vai ao encontro do propósito da computação ubíqua, isto é, o desenvolvimento de uma tecnologia que "desapareça".

Dentro do âmbito da personalização, este trabalho dá à estrutura de dados utilizada para armazenar as informações das entidades por uma aplicação o nome de Modelo de Entidade, enquanto o conjunto de informações referentes a uma entidade específica é nomeada Perfil de Entidade.

Este artigo está organizado da seguinte forma. Na Seção 2 é proposto o modelo do eProfile, bem como sua forma de operação. A Seção 3 apresenta um estudo de caso detalhando a utilização do eProfile. A Seção 4 descreve trabalhos relacionados, apresentando as contribuições deste modelo. Por fim, a Seção 5 apresenta as considerações finais.

2 Modelo

Pesquisas apontem que é desejável que os sistemas do futuro se tornem cada vez mais invisíveis aos seus usuários [16], e que um sistema só pode se tornar invisível se for pró-ativo, e só pode ser pró-ativo se conhecer o usuário [9]. Para que um sistema possa conhecer uma entidade, ele deve conhecer o histórico da entidade, compreender o contexto em que ela está inserido, bem como os dados coletados a respeito dela no mesmo contexto (ou num contexto similar), e ser capaz de processar este conjunto de informações em um perfil de entidade.

A Figura 1 apresenta a visão geral do eProfile, onde cada círculo representa um componente, e as setas indicam o fluxo dos dados.

O primeiro componente representado é a **aplicação**. A aplicação, dentro deste modelo, é qualquer aplicação que está sendo utilizada diretamente por uma entidade. Para que uma aplicação seja considerada apta a este modelo (isto é, possa utilizar as funcionalidades oferecidas pelo eProfile), ela precisa ser capaz de registrar suas informações em um

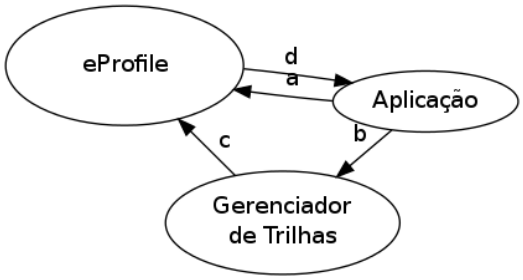

Figura 1: Modelo básico do eProfile

gerenciador de trilhas suportado pelo eProfile, e deverá seguir o protocolo de comunicação do eProfile para se comunicar com o mesmo. Seguidos estes pré-requisitos, a aplicação pode ser de qualquer tipo e operar em qualquer tipo de dispositivo que tenha comunicação com a internet.

O segundo componente representado é um **gerenciador de trilhas**. Um gerenciador de trilhas é capaz de registrar eventos realizados por uma entidade, colocando-os em uma ordem cronológica e registando informações relevantes ao evento (como, por exemplo, o contexto na qual o evento foi realizado). O modelo do eProfile é desenhado para oferecer suporte a qualquer gerenciador de trilhas, desde que o protocolo de comunicação entre o gerenciador de trilhas e o eProfile esteja codificadado no eProfile.

A Figura 1 apresenta ainda um terceiro serviço, o eProfile, que é o modelo apresentado neste trabalho. As setas indicam o fluxo dos dados. A comunicação tem início quando a aplicação registra-se no eProfile, enviando (i) o modelo de entidade utilizado pela aplicação e (ii) as regras para construção deste modelo (seta "a"). A partir deste momento, a aplicação passa a registrar os eventos no gerenciador de trilhas (seta "b"). O eProfile periodicamente busca esta informação a partir do gerenciador de trilhas, e constrói ou reconfigura o perfil de entidade (seta "c"). À medida que a aplicação tem necessidade de um perfil atualizado, ela busca este perfil no eProfile (seta "d").

Como o eProfile oferece suporte a várias aplicações, é possível que diferentes aplicações utilizem diferentes gerenciadores de trilhas. Por isso, o eProfile oferece também suporte à comunicação com vários gerenciadores de trilhas, segundo exemplificado na Figura 2.

A Figura 3 apresenta o modelo do eProfile. Os serviços "Aplicação" e "Gerenciador de Trilhas" são os mesmos mostrados na Figura 1, enquanto o serviço "eProfile" foi expandido para mostrar seu funcionamento interno.

Cada aplicação tem a possibilidade de montar seu próprio modelo de entidade. Para que o eProfile processe os perfis baseados em um modelo de entidade, ele precisa de três informações: (i) a trilha da entidade, (ii) o modelo de entidade utilizado pela aplicação e (iii) as regras de inferência para o perfil. Isto é visível na Figura 3, onde o agente

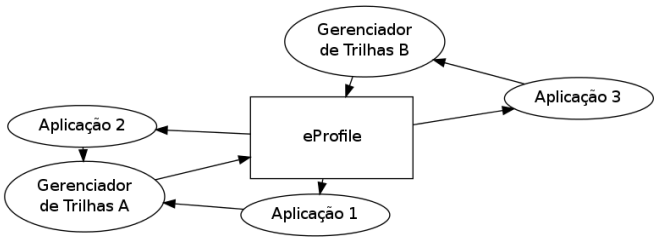

Figura 2: eProfile conectado a mais de um gerenciador de trilhas

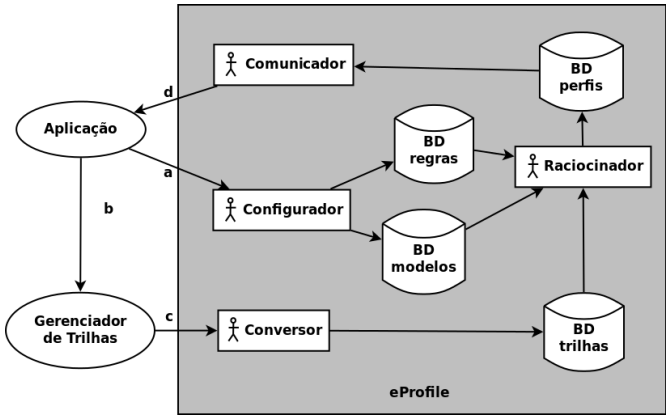

Figura 3: Modelo do eProfile

"Raciocinador" recebe dados destes três bancos. A trilha da entidade é carregada pelo eProfile a partir do gerenciador de trilhas, enquanto as outras duas informações são enviadas diretamente da aplicação para o eProfile. Desta forma, as regras de inferência definidas pela aplicação serão utilizadas para inferir o perfil da entidade.

As setas da Figura 3 indicam a direção do fluxo dos dados, onde podem se verificar dois pontos de entrada e um ponto de saída.

O primeiro ponto de entrada são as informações enviadas pela aplicação ao eProfile ou, mais especificamente, ao agente **configurador**. As informações que devem ser enviadas pela aplicação são as seguintes:

- o modelo de entidade utilizado pela aplicação, ou seja, o formato do perfil da entidade. Uma vez que a trilha da entidade seja processada, as informações inferidas serão armazenadas e disponibilizadas para a aplicação neste formato;

- as regras para inferência do perfil. Estas regras serão executadas em períodos regulares, de modo a atualizar o perfil da entidade. O configurador tem a função de receber as regras, validá-las e inseri-las na **base de regras**. Juntamente com as regras, é especificado a criticidade da informação. O sistema utilizará esta informação para determinar com que frequência a inferência desta informação será executada.

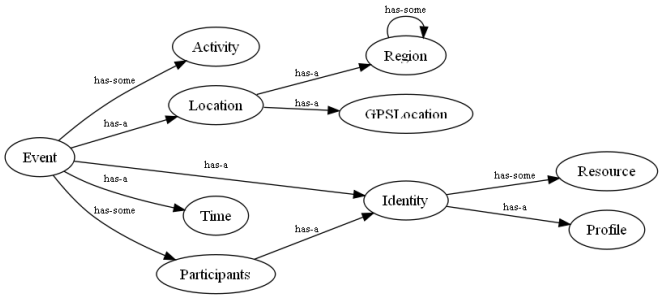

Figura 4: Ontologia de Trilha do eProfile

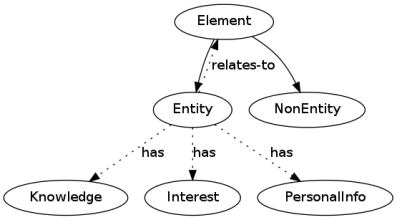

Figura 5: Ontologia de Perfil do eProfile

O segundo ponto de entrada é aquele através do qual são recebidas as trilhas das entidades, que são buscadas do gerenciador de trilhas pelo eProfile. Como o eProfile pode se conectar a diferentes gerenciadores de trilha (Figura 2), onde em cada um deles a trilha tem um formato específico, é necessário que seja realizada uma conversão entre o formato original e o formato utilizado pelo eProfile, para que as trilhas sejam armazenadas de forma padronizada. Este trabalho é realizado por um agente intitulado **conversor**, que é capaz de realizar a conversão para os formatos de trilha suportados pelo eProfile.

A trilha da entidade é então armazenada no eProfile em uma ontologia de trilha, chamada **eOntoTrail** (Figura 4). Esta ontologia possui as classes básicas para a construção de uma trilha, sendo que a mesma pode ser estendida de acordo com a especificação do gerenciador de trilhas que está sendo utilizado.

Uma vez de posse das trilhas, do modelo e das regras, o agente **raciocinador** realiza a inferência na trilha, gerando assim um perfil da entidade, que será uma extensão de uma ontologia de perfil básica denominada **eOntoProfile** (Figura 5). Esta ontologia, da mesma forma que a ontologia de trilha, contém classes básicas a partir das quais as aplicações vão especializar, utilizando-as para armazenar informações próprias. A aplicação pode então conectar-se ao eProfile e buscar o perfil de sua entidade, cujos dados são enviados pelo agente **comunicador**.

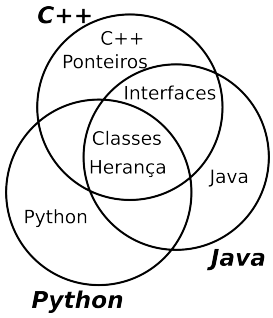

Figura 6: Módulos de estudo obrigatórios

3 Estudo de Caso

O estudo de caso apresentado é baseado na geração simulada de trilhas de estudantes dentro de um contexto de Educação Ubíqua. Educação Ubíqua pode ser compreendido como processos educacionais que ocorrem em qualquer lugar, a qualquer tempo e com qualquer dispositivo, de forma contínua, contextualizada e integrada ao cotidiano do aprendiz [1].

Foi decidido pela utilização de um cenário simplificado, de modo que o mesmo pudesse ser apresentado de forma aprofundada.

O cenário simula um curso de programação em linguagens orientadas a objeto. Deste curso participam três alunos (André, Pedro e Maria), cada um dos quais representado por uma entidade no eProfile.

O curso consiste em um conjunto de módulos relacionados a orientação a objetos. Destes módulos fazem parte conceitos básicos da matéria (classes, herança, *interfaces* e ponteiros) e linguagens de programação orientadas a objeto (C++, Python e Java). O estudante não é obrigado a concluir todos os módulos. No final do curso, no entanto, ele deve escolher a linguagem de programação que ele utilizará para responder as questões da prova final. Dependendo da linguagem que escolher, ele deve ter concluído, no mínimo, os módulos referentes àquela linguagem.

Por exemplo, caso o aluno decida realizar a prova em Java. Neste caso ele deve ter estudado classes, herança e *interfaces*, além do Java propriamente dito. Como a linguagem Java não contém o conceito de ponteiros, ele não é obrigado a ter cursado este módulo.

As obrigatoriedades dos módulos são representados pela Figura 6. No exemplo proposto, para que o estudante possa realizar a prova na linguagem Python, ele deve ter concluído no mínimo os módulos Classes, Herança e Python.

O ambiente é integrado a um sistema de educação ubíqua denominado LOCAL [2], utilizado para a distribuição contextualizada de objetos de aprendizagem (LO – *Learning Objects*). Os conteúdos dos módulos são distribuídos pelo LOCAL como objetos de aprendizagem, e o LOCAL registra em um gerenciador de trilhas cada vez que um módulo é iniciado ou concluído.

Cada evento (início ou conclusão de um objeto de aprendizagem) é registrado contendo as seguintes informações: (i) entidade que realizou o evento, (ii) objeto de aprendizagem, (iii) tipo de evento (início ou conclusão), (iii) data e hora e (iv) se é a primeira vez que o estudante realiza este tipo de evento.

O LOCAL registra no gerenciador de trilha não apenas os eventos, mas também as informações relacionadas àqueles eventos. Por exemplo, toda vez que um evento possui relação com um objeto de aprendizagem que está sendo mencionado pela primeira vez, é necessário que sejam incluídas todas as informações relevantes a este objeto, de modo que o eProfile possa utilizar estas informações ao montar a inferência.

Levando em conta o ambiente apresentado, a questão que o LOCAL deseja que o eProfile processe e registre no perfil da entidade é: esta entidade é proficiente em determinada linguagem? As subseções a seguir descrevem como ocorre este processo de inferência.

3.1 Ontologia

À medida que o registro das informações vai sendo realizada pelo LOCAL no gerenciador de trilhas, o eProfile vai solicitando informações atualizadas ao gerenciador de trilhas. A forma como o LOCAL e o gerenciador de trilhas se comunicam não é relevante ao eProfile, desde que o gerenciador de trilhas envie as informações em um formato que o eProfile seja capaz de interpretar.

De acordo com a Figura 3, o gerenciador de trilhas envia as informações ao módulo conversor do eProfile, que as converte para o formato OWL. As informações enviadas são de dois tipos: (i) as ocorrências, isto é, a trilha propriamente dita e (ii) as informações acessórias, que ajudarão o eProfile a montar a ontologia necessária para a inferência das informações. No cenário que está sendo descrito, um módulo concluído é um exemplo de ocorrência, enquanto as informações de dependências entre os módulos de aprendizagem são informações acessórias.

Tendo por base as informações enviadas pelo gerenciador de trilhas de acordo com o cenário descrito na Seção 3, o eProfile montou a ontologia mostrada na Figura 7. Cada círculo representa uma classe da ontologia.

É importante notar que cada uma das classes pai tem alguma representação na ontologia de trilha do eProfile, apresentada na Figura 4.

Algumas classes possuem informações que são necessárias para que o eProfile possa determinar se um estudante é proficiente em determinada linguagem. Um exemplo disto são os módulos referentes a conceitos básicos de orienta-

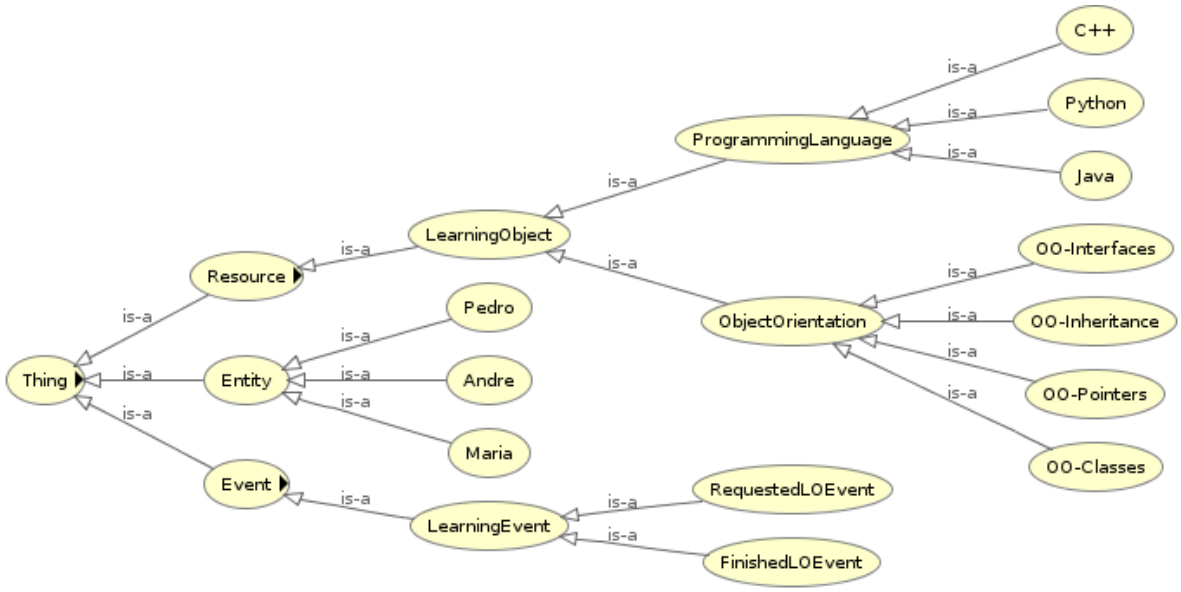

Figura 7: Ontologia do cenário apresentado

ção a objetos, que estão representados na ontologia como classes filhas da classe `ObjectOrientation`. A classe `OO-Interface`, por exemplo, diz respeito a um módulo que estuda o conceito de *interfaces*. Este conceito está presente nas linguagens C++ e Java, mas não em Python. Assim, esta classe é descrita da seguinte forma em OWL[1]:

```
1 Class: <el:OO-Interfaces>
2     SubClassOf:
3         <el:ObjectOrientation>,
4         <el:relatesTo> some <el:C++>,
5         <el:relatesTo> some <el:Java>
```

No exemplo, a linha 1 registra a classe, indicando que ela é filha da classe ObjectOrientation (linha 3, ver Figura 7). Além disso, as linhas 4 e 5 utilizam a propriedade `relatesTo` para indicar que o conceito de *interfaces* está relacionado às linguagens de programação (e, por consequência, às classes) `C++` e `Java`. Note-se que não é feita relação à classe `Python`.

Os eventos de requisição e conclusão do estudo de um objeto de aprendizado são registrados como classes filhas da classes `RequestedLOEvent` e `FinishedLOEvent` respectivamente que, por sua vez, são filhas da classe `LearningEvent`. Cada especialização da classe exige informações adicionais, de modo que a classe `FinishedLOEvent`. Este conjunto de classes é representado da seguinte forma:

```
1  Class: <el:Event>
2      SubClassOf:
3          Thing,
4          <el:hasEntity> some <el:Entity>,
5          <el:hasDateTime> exactly 1 xsd:dateTime,
6  Class: <el:LearningEvent>
7      SubClassOf:
8          <el:Event>,
9          <el:hasLearningObject> some
10                                 <el:LearningObject>
11 Class: <el:FinishedLOEvent>
12     SubClassOf:
13         <el:LearningEvent>,
14         <el:isFirstTime> exactly 1 xsd:boolean
```

Desta forma, cada evento registrado deve ser vinculado a uma entidade (linha 4) e ter uma data e hora (linha 5); um evento de aprendizagem deve ter um objeto de aprendizagem relacionado (linhas 9 e 10); e um evento de término de aprendizado deve ter registrado se o evento ocorreu pela primeira vez.

O código abaixo representa o registro de um evento, onde a entidade André (linha 4) concluiu um objeto de aprendizagem sobre Classes (linha 5) pela primeira vez (linha 7) em 08/07/2012 às 10h30 (linhas 8 e 9). Note-se que, como esta classe é filha da classe `FinishedLOEvent`, as propriedades requisitadas na classe estão sendo informadas nas classes filhas.

[1]Embora o eProfile utilize internamente a sintaxe OWL/RDF [14], a sintaxe Manchester-OWL está sendo usada neste artigo por ser mais compacta a amigável [13].

```
Class: <el:FinishedLOEvent1>
    SubClassOf:
        <el:FinishedLOEvent>,
        <el:hasEntity> some <el:Andre>,
        <el:hasLearningObject> some
                                    <el:OO-Classes>
        <el:isFirstTime> value true,
        <el:hasDateTime> value
                              "2012-07-08T10:30:00"
```

3.2 Regras

Para adicionar ao perfil de um aluno se ele é proficiente em uma determinada linguagem (que é o objetivo do cenário, de acordo com a Seção 3, os seguintes passos devem ser realizados:

1. Identificar os objetos de aprendizados requeridos para a proficiência na linguagem;
2. Identificar os objetos de aprendizagem concluídos pelo estudante referentes ao item 1;
3. Verificar se todos os objetos de aprendizagem necessários foram concluídos.

Neste exemplo, será verificado se o estudante é proficiente na linguagem Java. Para tal, ele deve ter concluído os objetos de aprendizagens que fazem relação com a linguagem Java (neste caso, classes, herança em *interfaces*), bem como o objeto de aprendizado da linguagem Java. Na ontologia, isto refere-se à criação de uma nova classe (`JavaRequirement`) que seja equivalente à classe `Java` (que é filha da classe de `LearningObject` e, portanto, um objeto de aprendizado, ou quaisquer outras classes cuja propriedade `relatesTo` aponte para a classe `Java`. Assim, esta regra fica definida da seguinte forma:

```
Class: <el:JavaRequirement>
    EquivalentTo:
        <el:Java>
         or (<el:relatesTo> some <el:Java>)
```

O segundo passo é a identificação dos objetos de aprendizagem concluídos pelo estudante relacionados ao Java. Para tal, é necessário que:

1. os eventos sejam do tipo `FinishedLOEvent` (só é importante saber os objetos de aprendizado que o estudante concluiu, e não os que ele iniciou);
2. os eventos tenham sido realizados pela entidade específica;
3. os eventos sejam relacionados ao Java, ou seja, filhas da classe `JavaRequirement` criada acima;
4. os eventos tenham sido realizados pela primeira vez (somente é importante saber os objetos de aprendizado concluídos pelo aluno, e não quantas vezes).

Para que o eProfile possa identificar estes eventos para a entidade Andre, a seguinte regra é enviada ao eProfile:

```
Class: <el:JavaLessonsTaken>
    EquivalentTo:
        <el:FinishedLOEvent>
         and (<el:hasEntity> some <el:Andre>)
         and (<el:hasLearningObject> some
                              <el:JavaRequirement>)
         and (<el:isFirstTime> value true)
```

Uma vez que estas duas regras estejam criadas na ontologia, o eProfile irá executar o raciocinador, que gerará uma nova ontologia OWL, com as classes inferidas identificadas como filhas das classes criadas.

A partir da execução do raciocinador, uma regra de atualização de perfil é criada, onde são comparados o número de módulos Java com o número de objetos de aprendizagem relacionados ao Java que foram concluídos pela entidade. Se o número for igual, o usuário é proficiente. Esta regra é definida da seguinte forma:

```
JavaProficience <=
  (Count(SubClasses(JavaRequirement))) equals
          (Count(SubClasses(JavaLessonsTaken)))
```

4 Trabalhos Relacionados

Atualmente, as áreas de computação ubíqua, sensibilidade a contexto, trilhas e personalização são bastante promissoras, sendo que um grande número de trabalhos são publicados anualmente a este respeito. No entanto, não foi encontrado nenhum trabalho que faça uso de inferência em trilhas com o objetivo de personalizar a experiência do usuário dentro de um domínio genérico. Assim, o modelo apresentado neste trabalho busca preencher esta lacuna.

O PeLeP [6] suporta aperfeiçoamento automático do perfil de um aprendiz em ambientes de educação ubíqua, utilizando o histórico dos aprendizes para a atualização dos perfis através de inferências. O eProfile utiliza uma estratégia semelhante. No entanto, as regras de inferência são definidas pela aplicação, e não pelo modelo. Esta alteração é necessária para que a inferência passe de um domínio específico (educação) para um domínio genérico. Além disso,

Trabalho	Interoperabilidade	Perfis dinâmicos	Armazenamento	Sensibilidade a contexto	Utiliza trilhas	Domínio
PeLeP	Não	Sim	Servidor	Sim	Parcial	Educação
Chellouche et al	Sintática	Sim	Servidor	Sim	Não	Serviços multimídia
UUCM	Sintática	Sim	Cliente	Sim	Não	Genérico
eProfile	Semântica	Sim	Servidor	Sim	Sim	Genérico

Tabela 1: Comparação entre os trabalhos apresentados

ao contrário do eProfile, as regras são dinâmicas (uma aplicação pode definir ou remover regras dinamicamente), ao invés de estarem pré-definidas no modelo. Isto permite a evolução e refino das regras de inferência.

O modelo apresentado por Chellouche et al [3], assim como o eProfile, apresenta um perfil unificado que pode ser utilizado por diversas aplicações. No entanto, ao contrário do eProfile, o modelo assume a interoperabilidade sintática, o que significa que as todas aplicações precisam registrar os eventos utilizando o mesmo formato, o que pode fazer com que a aplicação não consiga registrar o evento com a riqueza de detalhes necessária. Além disso, assim como o PeLeP, o modelo de Chellouche et al [3] está limitado a um domínio específico, que são os serviços multimídia.

O modelo UUCM [8], ao contrário dos outros modelos, trabalha em um domínio genérico. Ele faz isso através da criação de uma ontologia básica que pode ser estendida pelas aplicações. Ao contrário das outras propostas (e do eProfile), o UUCM armazena os perfis no cliente, ao invés do servidor. Isto traz vantagens e desvantagens. Entre as vantagens pode-se citar a privacidade e acesso desconectado. No entanto, esta abordagem limita a interoperabilidade entre sistemas, uma vez que cada cliente fica com uma versão do próprio perfil, restringindo a utilização de um perfil unificado.

A Tabela 1 exibe um comparativo entre o eProfile e os trabalhos relacionados apresentados.

O eProfile fez utilização das ideias apresentadas nos trabalhos, como o aperfeiçoamento automático de perfis, a utilização de perfis unificados e o funcionamento em um domínio genérico. A estas ideias, o eProfile traz como inovação a utilização de trilhas para a extração de perfis, o que facilita a inferência com sensibilidade a contexto. Além disso, o eProfile permite que cada aplicação registre sua trilha no formato que considerar mais proveitoso para si, permitindo assim que a riqueza de detalhes registrados por uma aplicação possa se traduzir em um perfil mais exato da entidade.

5 Conclusão

Apresentamos neste artigo o eProfile como um modelo para suporte ao gerenciamento de perfis em ambientes ubíquos. O eProfile permite a inferência de trilhas de entidades através da definição de regras pelas aplicações. Foi realizado um experimento onde foi simulada a inferência de informações a respeito de estudantes em um ambiente de Educação Ubíqua. As principais contribuições do modelo são: (i) a utilização de trilhas para extração de perfis e (ii) a interoperabilidade semântica, que permite que as diferentes aplicações registrem suas trilhas utilizando uma ontologia própria e permitindo com que as aplicações trabalhem em um domínio genérico, isto é, sem um contexto preestabelecido.

O eProfile é uma proposta inicial e requer aperfeiçoamento, especialmente na expansão dos cenários. O cenário apresentado mostra o eProfile trabalhando dentro de um domínio específico e, desta forma, não demonstra como múltiplas aplicações trabalhando em contextos diferentes podem gerar um perfil único. Desta forma, trabalho futuro neste modelo será realizado nesta direção.

Referências

[1] D. Barbosa, C. Geyer, and J. Barbosa. Globaledu - an architecture to support learning in a pervasive computing environment. In A. Rettberg and C. Bobda, editors, *New Trends and Technologies in Computer-Aided Learning for Computer-Aided Design*, volume 192 of *IFIP International Federation for Information Processing*, pages 1–10. Springer Boston, 2005.

[2] J. Barbosa, R. Hahn, S. Rabello, and D. Barbosa. Local: a model geared towards ubiquitous learning. *SIGCSE Bull.*, 40:432–436, March 2008.

[3] S. A. Chellouche, J. Arnaud, and D. Négru. Flexible user profile management for context-aware ubiquitous environments. In *Proceedings of the 7th IEEE conference on Consumer communications and networking conference*, CCNC'10, pages 980–984, Piscataway, NJ, USA, 2010. IEEE Press.

[4] A. Dey. Understanding and using context. *Personal and Ubiquitous Computing*, 6(1):4–7, 2001.

[5] D. Heckmann. *Ubiquitous User Modeling*. PhD thesis, Department of Computer Science, Saarland University, Germany, November 2005.

[6] D. Levis, J. L. V. Barbosa, S. C. C. S. Pinto, and D. N. F. Barbosa. Aperfeiçoamento automático do perfil do aprendiz em ambientes de educação ubíqua. *Revista Brasileira de Informática na Educação*, 16(1), 2008.

[7] M. D. Mulvenna, S. S. Anand, and A. G. Büchner. Personalization on the net using web mining: introduction. *Commun. ACM*, 43:122–125, August 2000.

[8] C. Niederée, A. Stewart, B. Mehta, and M. Hemmje. A Multi-Dimensional, Unified User Model for Cross-System Personalization. In *E4PIA Workshop 2004*, 2004.

[9] M. Satyanarayanan. Pervasive computing: Vision and challenges. *IEEE Personal Communications*, 8:10–17, 2001.

[10] J. Silva, J. Rosa, J. Barbosa, D. Barbosa, and L. Palazzo. Content distribution in trail-aware environments. *Journal of the Brazilian Computer Society*, 16:163–176, 2010. 10.1007/s13173-010-0015-1.

[11] J. Silva, J. Rosa, J. Barbosa, D. N. F. Barbosa, and L. A. M. Palazzo. Content distribution in trial-aware environments. In *Proceedings of the XV Brazilian Symposium on Multimedia and the Web*, WebMedia '09, pages 15:1–15:8, New York, NY, USA, 2009. ACM.

[12] M. Viviani, N. Bennani, and E. Egyed-Zsigmond. A Survey on User Modeling in Multi-Application Environments. In *The Third International Conference on Advances in Human-oriented and Personalized Mechanisms, Technologies, and Services CENTRIC'10*, pages 111–116. IEEE, Aug. 2010.

[13] W3C. OWL 2 web ontology language manchester syntax, 2009. `http://www.w3.org/TR/owl2-manchester-syntax/`. Acessado em 08/07/2012.

[14] W3C. OWL web ontology language guide, 2011. `http://www.w3.org/TR/owl-guide/`. Acessado em 05/05/2012.

[15] M. Weiser. The computer for the 21st century. *Scientific America*, pages 94–104, 1991.

[16] M. Weiser and J. S. Brown. Designing calm technology. *PowerGrid Journal*, July 1996.